

RAPPORT TECHNIQUE

Conception, architecture et fonctionnement

PENTEST RAG

Assistant d'exploitation par génération augmentée de récupération

« Transformer une base de connaissances offensive en guidage d'exploitation structuré, précis et exécutable »

Systeme RAG auto-hébergé et conteneurisé, spécialisé pour le pentest

Intégration native avec l'agent autonome Strix

Stack : FastAPI · Qdrant · LangChain · Sentence-Transformers · Docker Compose

LLM : Ollama (local) / OpenRouter (cloud)

Rapport rédigé le 18 juillet 2026

Résumé

Le projet Pentest RAG est un système de génération augmentée de récupération (Retrieval-Augmented Generation) auto-hébergé et conteneurisé, spécialement conçu pour assister les opérations de sécurité offensive. Il transforme une base de connaissances locale de ressources de pentest (CVE, writeups, code d'exploit, documentation d'outils) en un assistant interactif capable de fournir un guidage d'exploitation structuré, actionnable et techniquement exact. En ancrant chaque réponse dans des données vérifiées, le système réduit fortement le risque d'hallucination des grands modèles de langage et accélère la phase de développement d'exploits.

Ce rapport détaille l'architecture du système (FastAPI, base vectorielle Qdrant, embeddings nomic-embed-text, reranking par Cross-Encoder, backend LLM Ollama/OpenRouter, orchestration Docker Compose) et son pipeline en trois phases — ingestion, récupération intelligente avec reclassement, et génération structurée en JSON. Il documente également l'API REST, l'intégration avec l'agent autonome Strix, le moteur d'exploitation à trois niveaux (Metasploit natif, SearchSploit, exécution directe des payloads du RAG) avec sa barrière de validation humaine, ainsi que le déploiement, l'évaluation et les considérations de sécurité.

Mots-clés :

RAG, pentest, exploitation, Qdrant, embeddings, reranking, LLM, FastAPI, Docker, Strix, Metasploit.

Table des matières

Clic droit → « Mettre à jour les champs » pour générer la table.

Liste des figures

- Figure 2.1 — Architecture technique du Pentest RAG
- Figure 3.1 — Pipeline en trois phases
- Figure 5.1 — Chaîne de récupération et de reclassement
- Figure 8.1 — Intégration avec l'agent Strix
- Figure 9.1 — Moteur d'exploitation à trois niveaux

Introduction

La phase d'exploitation d'un test d'intrusion exige une connaissance fine et à jour des vulnérabilités, des payloads et des techniques associées. Traditionnellement, le testeur consacre un temps considérable à parcourir ses notes, GitHub, Exploit-DB ou les moteurs de recherche pour retrouver l'information exacte relative à un service et une version donnés. Les grands modèles de langage (LLM) promettent d'accélérer cette recherche, mais souffrent d'un défaut rédhibitoire dans un contexte offensif : l'hallucination — la production de CVE inexistantes, de syntaxes de payload erronées ou de techniques inapplicables.

Le projet Pentest RAG répond à ce problème en appliquant le paradigme de la génération augmentée de récupération : plutôt que de s'en remettre à la seule mémoire interne du modèle, le système ancre chaque réponse dans un corpus vérifié de connaissances offensives. Le résultat est un guidage d'exploitation fiable, livré dans un format directement exploitable par un opérateur ou par un agent automatisé comme Strix. Ce rapport présente une analyse technique complète du projet : de l'architecture conteneurisée au pipeline de traitement, en passant par l'API, l'intégration Strix et le moteur d'exploitation.

Chapitre 1 — Présentation générale

1.1 Objectif et positionnement

Le Pentest RAG n'est pas un assistant généraliste : il est spécialisé pour l'exploitation. Son unique vocation est de transformer une base de connaissances offensive en réponses « armées » — payloads réels, commandes exactes, scripts complets — plutôt qu'en explications théoriques. Il s'inscrit dans la chaîne outillée d'un pentest et cible spécifiquement l'accélération et la fiabilisation de la phase d'exploitation.

1.2 Principe de fonctionnement

Le système suit un pipeline en trois phases : (1) l'ingestion construit une base vectorielle à partir de documents bruts ; (2) la récupération retrouve, pour une requête donnée, les fragments les plus pertinents ; (3) la génération produit, à partir de ce contexte, une réponse structurée en JSON contenant les éléments d'exploitation. L'ensemble est exposé via une API REST et orchestré par Docker Compose.

1.3 Atouts clés

- **Réduction des hallucinations** — en forçant le modèle à s'appuyer sur un contexte récupéré d'une base de confiance, le système réduit fortement le risque de conseils d'exploitation fabriqués ou erronés.
- **Sortie structurée et actionnable** — un schéma JSON strict (`vulnerability_id`, `payloads`, `exploit_code...`) rend la réponse directement consommable par des scripts ou des agents, sans analyse de prose.
- **Précision orientée cible** — le pipeline de scoring pondère les résultats selon le contexte de la cible (service, version, OS, findings).
- **Traitement robuste des connaissances** — déduplication, nettoyage du JSON bruité, OCR des images, filtrage du bruit binaire, préservation du formatage critique.
- **Portabilité** — déploiement en une commande via Docker Compose, persistance par volumes nommés.
- **Backend LLM flexible** — OpenRouter (cloud) ou Ollama (local) selon les besoins de performance, de coût ou de confidentialité.
- **Optimisé pour la vitesse** — cache de requêtes, recherche vectorielle efficace, instance RAG en singleton pour éviter le rechargement des modèles.

Chapitre 2 — Architecture technique

2.1 Vue d'ensemble

L'architecture est entièrement conteneurisée et orchestrée par Docker Compose. Elle réunit deux services principaux sur un réseau privé (pentest_net) : le service applicatif pentest-rag (API FastAPI) et la base vectorielle Qdrant. Le backend LLM (Ollama ou OpenRouter) est sollicité en externe, et la persistance est assurée par des volumes montés.

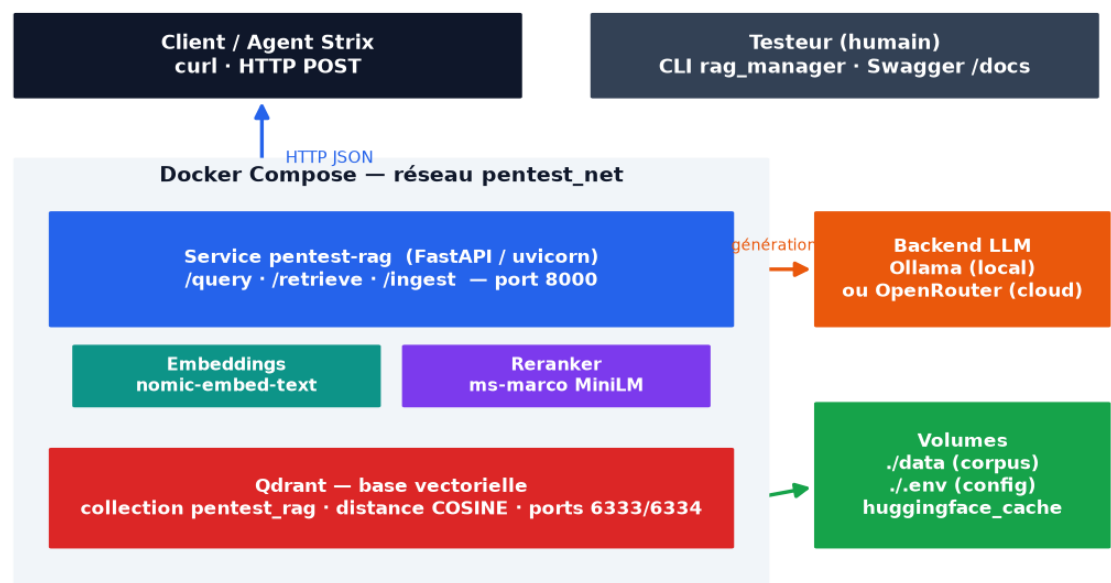


Figure 2.1 — Architecture technique du Pentest RAG.

2.2 Composants

Composant	Rôle	Technologie
API applicative	Exposer les endpoints /query, /retrieve, /ingest	FastAPI + uvicorn (port 8000, repli 8001)
Base vectorielle	Stockage et recherche sémantique des chunks	Qdrant (ports 6333/6334, distance COSINE)
Embeddings	Vectorisation des textes et des requêtes	nomic-ai/nomic-embed-text-v1 (HuggingFace)
Reranker	Reclassement fin des candidats	cross-encoder/ms-marco-MiniLM-

		L-6-v2
Backend LLM	Génération de la réponse structurée	Ollama (local) / OpenRouter (cloud)
Orchestration	Déploiement et réseau	Docker Compose (réseau pentest_net)
Persistence	Corpus, configuration, cache modèles	Volumes ./data, ./env, huggingface_cache

Tableau 2.1 — Composants de l'architecture.

2.3 Choix de conception

- **Singleton RAG** — la classe PentestRAG est instanciée une seule fois au démarrage, évitant le rechargement coûteux des modèles d'embedding et de reranking.
- **Détection de l'environnement** — la configuration s'adapte automatiquement à l'exécution en conteneur (résolution des hôtes qdrant et host.docker.internal).
- **Cycle de vie de Qdrant automatisé** — hors Docker, un utilitaire démarre ou crée le conteneur Qdrant et attend qu'il soit prêt avant toute opération.

Chapitre 3 — Le pipeline en trois phases

Le cœur fonctionnel du système repose sur un pipeline en trois phases, toutes adossées à la base vectorielle Qdrant.



Qdrant (base vectorielle) — socle partagé entre les trois phases

Figure 3.1 — Pipeline en trois phases : ingestion, récupération, génération.

- **Ingestion** — collecte, nettoyage, découpage et vectorisation des données, puis stockage dans Qdrant (détaillé au chapitre 4).
- **Récupération + reranking** — recherche des fragments pertinents, reclassement par Cross-Encoder et scoring composite (chapitre 5).
- **Génération** — production d'une réponse structurée en JSON à partir du contexte récupéré et du contexte cible (chapitre 6).

Chapitre 4 — Ingestion et construction de la base

4.1 Objectif

L'ingestion transforme des données brutes hétérogènes (exploits, CVE, writeups, documentation, images) en vecteurs exploitables, indexés dans Qdrant. Le pipeline est conçu pour absorber le désordre du monde réel tout en maximisant la qualité du corpus.

4.2 Formats pris en charge

Le module d'ingestion reconnaît un large éventail d'extensions textuelles (.txt, .md, .json, .csv, .xml, .yaml...) et de fichiers de code (.py, .js, .php, .sh, .sql, .nse...). Les images (.png, .jpg, .tiff...) sont traitées par reconnaissance optique de caractères (OCR) via Tesseract, ce qui permet d'extraire le texte de captures d'écran de writeups.

4.3 Nettoyage et qualité des données

- **Nettoyage du texte** — suppression des caractères non imprimables et des espaces multiples, tout en préservant les sauts de ligne indispensables à la structure des exploits.
- **Nettoyage du JSON** — extraction sélective des champs utiles (cveid, title, description, payload, poc...) et rejet des clés bruitées (references, tags, metrics, timeline...), afin de densifier les données CVE.
- **Filtrage du bruit binaire** — les fragments à faible proportion de caractères imprimables (seuil 80 %) sont écartés.
- **Filtrage des fragments faibles** — les chunks de moins de 30 caractères sont ignorés.

4.4 Déduplication multi-niveaux

Deux niveaux de déduplication par empreinte SHA-256 garantissent l'unicité du corpus : au niveau du fichier (rejet des documents identiques) et au niveau du chunk (rejet des fragments redondants). Cette approche évite de sur-représenter certaines connaissances lors de la récupération.

4.5 Découpage et vectorisation

Les documents sont découpés par un RecursiveCharacterTextSplitter, puis chaque fragment est vectorisé et inséré dans Qdrant. L'insertion (upsert) est robuste : traitement par lots, délais d'attente configurables et réessais avec repli exponentiel en cas de dépassement de délai.

Paramètre	Valeur	Rôle
CHUNK_SIZE	500	Taille cible d'un fragment (caractères)
CHUNK_OVERLAP	200	Chevauchement entre fragments

		consécutifs
upsert_batch_size	16	Fragments par lot d'insertion Qdrant
upsert_timeout	180 s	Délai d'attente d'une insertion
upsert_retries	6	Réessais en cas de dépassement de délai
Distance	COSINE	Métrique de similarité vectorielle

Tableau 4.1 — Paramètres d'ingestion.

Chapitre 5 — Récupération intelligente et reclassement

5.1 Objectif

La phase de récupération vise à sélectionner, parmi des milliers de fragments, le sous-ensemble le plus pertinent pour la requête et la cible. Elle combine recherche sémantique, reclassement neuronal et scoring heuristique.

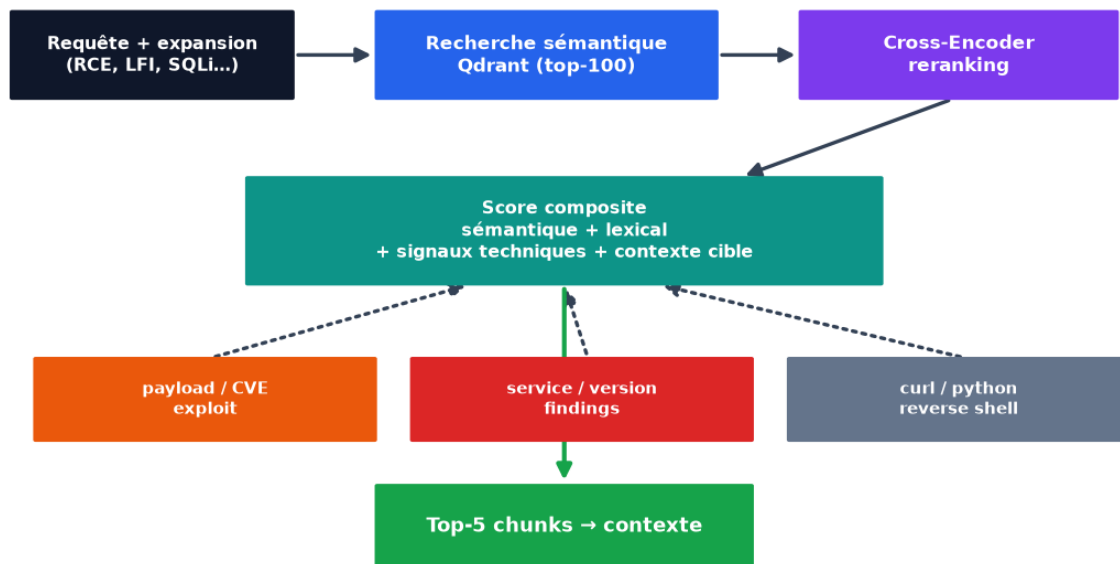


Figure 5.1 — Chaîne de récupération et de reclassement.

5.2 Étapes

- **Expansion de requête** — les acronymes sont enrichis (RCE → Remote Code Execution, LFI, SQLi, XSS...) et le contexte cible (service, version, OS, findings) est concaténé à la requête.
- **Recherche sémantique large** — Qdrant retourne un premier ensemble de candidats (récupération à large top-k).
- **Reclassement Cross-Encoder** — le modèle ms-marco-MiniLM-L-6-v2 attribue un score de pertinence fin à chaque paire (requête, fragment).
- **Scoring composite** — le score final agrège la pertinence sémantique, les correspondances lexicales, les signaux techniques (payload, CVE, curl, reverse shell...) et la correspondance au contexte cible.
- **Repli ciblé** — si trop peu de fragments correspondent à la cible, une requête de repli enrichie (« exploit writeup poc payload... ») complète les candidats.
- **Extraction des extraits utiles** — des documents longs, seules les sections les plus pertinentes sont retenues, en privilégiant les passages contenant CVE, payloads et exploits.

5.3 Paramètres de récupération

Paramètre	Valeur	Rôle
RETRIEVAL_TOP_K	100	Candidats issus de la recherche sémantique
RERANK_TOP_K	5	Fragments retenus après reclassement
CONTEXT_PER_DOC_MAX_CHARS	8000	Longueur maximale conservée par document
CONTEXT_MAX_CHARS	24000	Taille maximale du contexte total
CONTEXT_SECTION_MAX_CHARS	2200	Longueur maximale d'une section extraite

Tableau 5.1 — Paramètres de récupération.

5.4 Cache de requêtes

Un cache en mémoire (avec durée de vie d'une heure et limite de 1000 entrées, éviction du plus ancien) mémorise les réponses par clé combinant la requête et le contexte cible. Il accélère les requêtes répétées et évite un appel LLM redondant.

Chapitre 6 — Génération structurée

6.1 Principe

La génération transforme le contexte récupéré en un plan d'exploitation. Le modèle est contraint, par un prompt d'ingénierie strict, à se comporter comme une « API lisible par machine » et à ne produire qu'un unique objet JSON conforme à un schéma imposé — sans prose ni markdown superflus.

6.2 Schéma de réponse (StructuredRAGResponse)

Champ	Description
vulnerability_id	Identifiant précis (CVE-XXXX-XXXX ou nom standard de la vulnérabilité)
reasoning	Explication technique du fonctionnement et de l'applicabilité de la faille
actionable_commands	Commandes ordonnées pour vérifier et déclencher l'exploit
payloads	Tableau des chaînes de payload brutes (curl, versions encodées, reverse shells)
exploit_code	Script complet (Python/Bash/C) ou bloc de payload multi-lignes
post_exploitation	Étapes de stabilité, persistance et élévation de privilèges
confidence_score	Score de fiabilité entre 0.0 et 1.0

Tableau 6.1 — Schéma de la réponse structurée.

6.3 Garde-fous

- **Contexte insuffisant** — si la base ne retourne pas assez de contexte, le système renvoie explicitement une réponse « Insufficient Context » avec un score de confiance nul, plutôt que d'inventer un exploit.
- **Sortie JSON forcée** — format JSON imposé côté fournisseur (response_format json_object pour OpenRouter, format json pour Ollama) et nettoyage/analyse défensive côté serveur.
- **Paramètres de génération** — température basse (0.1) et plafond de jetons (2048) pour favoriser la précision et la reproductibilité.

6.4 Backends LLM

Le fournisseur est sélectionné via la variable `LLM_PROVIDER`. En mode OpenRouter, l'appel est authentifié par clé et cible l'API de complétion de chat ; en mode Ollama, l'appel est adressé au serveur local (avec résolution de l'hôte adaptée à Docker). Le modèle est configurable (par ex. `google/gemma-4-31b-it` en cloud, ou un modèle local équivalent).

Chapitre 7 — API REST et endpoints

L'API FastAPI expose trois points d'entrée, documentés automatiquement via Swagger (<http://localhost:8000/docs>).

Endpoint	Méthode	Rôle
<code>/query</code>	POST	Pipeline complet : récupération + génération du plan d'exploitation structuré
<code>/retrieve</code>	POST	Voie rapide : récupération du contexte brut, sans appel LLM
<code>/ingest</code>	POST	Déclenche l'ingestion et l'indexation du dossier data/

Tableau 7.1 — Endpoints de l'API.

7.1 Corps de requête

Les endpoints `/query` et `/retrieve` partagent un même format de requête : une chaîne `query`, un objet optionnel `target_context` (`service`, `version`, `os`, `findings`), un paramètre `k` (nombre de fragments) et un champ optionnel `model`.

```
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{
    "query": "Exploit Apache 2.4.49 path traversal for RCE",
    "target_context": {
      "service": "Apache", "version": "2.4.49",
      "os": "Linux", "findings": ["Port 80 open"]
    }
  }'
```

7.2 Interface en ligne de commande

Un gestionnaire interactif (`rag_manager.py`) offre un menu pour interroger le RAG, effectuer une récupération rapide, lancer le serveur, collecter des données depuis des liens web, nettoyer le corpus, lancer l'ingestion, changer de modèle IA et vérifier l'état de Qdrant. Le CLI de `main.py` expose par ailleurs les commandes `--ingest`, `--query` et `--serve`.

Chapitre 8 — Intégration avec l'agent Strix

8.1 Contrat d'intégration

L'intégration avec l'agent autonome Strix est explicitement définie par le fichier `strix_instructions.md`, qui fournit un contrat éprouvé. Strix construit un `target_context` à partir de sa reconnaissance, puis interroge le RAG comme un « module d'exploit » de confiance.



Figure 8.1 — Intégration avec l'agent Strix.

8.2 Déroulement

- Strix effectue la reconnaissance et détecte le service et sa version.
- Il appelle d'abord `POST /retrieve` pour vérifier l'existence de connaissances pertinentes.
- Une barrière de pertinence structurelle est appliquée : le contexte doit dépasser 100 caractères et compter au moins 3 lignes non vides ; sinon, l'appel à `/query` est évité.
- Si la barrière est franchie, Strix appelle `POST /query` pour obtenir le plan structuré.
- Strix exécute les `actionable_commands` et `payloads`, puis exploite le `post_exploitation`.

8.3 Résilience et seuils de décision

Le contrat spécifie un délai d'attente obligatoire de 120 secondes, une logique de réessai (bascule vers l'hôte suivant) et une procédure de repli (poursuite sans RAG en cas d'échec). Le champ `confidence_score` guide la décision : exploiter directement ($> 0,7$), vérifier d'abord ($0,4-0,7$) ou approfondir la reconnaissance ($< 0,4$).

Chapitre 9 — Moteur d'exploitation à trois niveaux

9.1 Principe

Le module metasploit_bridge.py constitue un moteur d'exploitation à repli hiérarchique. À partir du plan produit par le RAG, il choisit la voie d'exécution la plus fiable disponible, selon trois niveaux (tiers) de priorité décroissante.

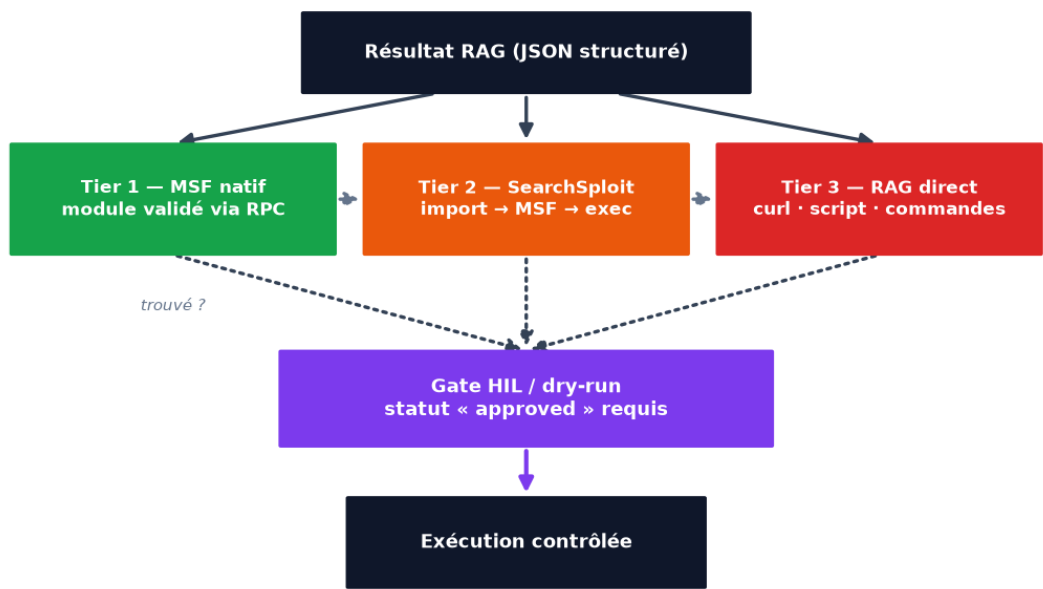


Figure 9.1 — Moteur d'exploitation à trois niveaux avec barrière de validation.

Niveau	Condition	Action
Tier 1 — MSF natif	Le module existe dans Metasploit	Validation puis exécution via MSF RPC (pymetasploit3)
Tier 2 — SearchSploit	Module absent de MSF mais trouvé via SearchSploit	Import du module dans MSF (~/.msf4/modules/...), rechargement puis exécution
Tier 3 — RAG direct	Ni MSF ni SearchSploit	Exécution directe des payloads HTTP (curl), du script d'exploit, puis des commandes

Tableau 9.1 — Les trois niveaux d'exécution.

9.2 Barrière de validation humaine (HIL)

Quel que soit le niveau retenu, aucun exploit n'est exécuté sans franchir une barrière de validation. Chaque plan (ExploitPlan) démarre au statut PENDING_APPROVAL et doit passer explicitement au statut APPROVED avant toute exécution ; à défaut, il est bloqué. Cette barrière « human-in-the-loop » (dry-run) garantit qu'aucune action offensive n'est déclenchée de manière incontrôlée.

9.3 Substitution des variables

Les espaces réservés (TARGET_IP, ATTACKER_IP, RHOST, LHOST, RPORT, LPORT...) sont résolus au moment de l'exécution avec les valeurs réelles de la cible et de l'attaquant, dans les payloads, les scripts et les commandes.

Chapitre 10 — Déploiement et configuration

10.1 Démarrage

Le système démarre en une commande via Docker Compose. Le premier lancement est plus long car les images (Qdrant + RAG) sont téléchargées et les modèles de ML sont récupérés dans le conteneur. La base Qdrant est préchargée, ce qui dispense l'utilisateur de toute ingestion.

```
docker compose up --build
# Vérification : http://localhost:8000/docs
```

10.2 Configuration (.env)

La configuration se fait par un fichier .env :

```
LLM_PROVIDER=openrouter
OPENROUTER_API_KEY=VOTRE_CLE
LLM_MODEL=google/gemma-4-31b-it
EMBEDDING_MODEL=nomic-ai/nomic-embed-text-v1
QDRANT_URL=http://qdrant:6333
QDRANT_COLLECTION_NAME=pentest_rag
```

10.3 Règles de connexion

Contexte	URL
Hôte → RAG	http://localhost:8000
Conteneur → RAG	http://pentest-rag:8000
RAG → Qdrant	http://qdrant:6333

Tableau 10.1 — Règles de connexion réseau.

10.4 Débogage

Les journaux se consultent via « docker compose logs -f pentest-rag ». Les points d'attention principaux sont le temps de démarrage initial (téléchargement des modèles) et la validité de la clé API OpenRouter.

Chapitre 11 — Évaluation et performances

Un script de benchmark (`scripts/benchmark_rag.py`) mesure la qualité et la latence du système sur un jeu de requêtes de référence. Pour chaque cas, il compare la réponse au résultat attendu et calcule un score de précision, ainsi que les temps de récupération et de génération.

11.1 Métriques

- **Précision par identifiant** — correspondance (souple) du `vulnerability_id` attendu (40 % du score).
- **Précision par mots-clés** — proportion des mots-clés attendus retrouvés dans le raisonnement et les commandes (60 % du score).
- **Latence de récupération** — temps de la phase de recherche + reranking.
- **Latence de génération** — temps de l'appel au LLM.
- **Latence totale et P95** — temps de bout en bout, et 95^e centile.

Ce cadre d'évaluation permet de comparer objectivement des configurations (modèles d'embedding, backends LLM, paramètres de récupération) et de suivre les régressions.

Chapitre 12 — Forces, limites et considérations éthiques

12.1 Forces

- Ancrage systématique des réponses dans une base vérifiée (anti-hallucination).
- Sortie machine directement exploitable par des agents (contrat JSON strict).
- Récupération orientée cible et sensible aux signaux techniques.
- Déploiement portable et reproductible (Docker Compose, volumes persistants).
- Repli d'exploitation robuste (Metasploit → SearchSploit → RAG direct).

12.2 Limites et points d'attention

- Qualité des réponses dépendante de la couverture et de la fraîcheur du corpus ingéré.
- Dépendance au backend LLM (coût/latence en cloud, ressources matérielles en local).
- Le démarrage initial est long (téléchargement des modèles).
- La pertinence du reranking dépend de l'adéquation du modèle Cross-Encoder au domaine.

12.3 Considérations éthiques et de sécurité

Le Pentest RAG produit des capacités offensives réelles (payloads, exploits). Son usage doit être strictement cantonné à des tests autorisés, dans un cadre légal et contractuel explicite (règles d'engagement). La barrière de validation humaine du moteur d'exploitation constitue à cet égard un garde-fou essentiel : elle impose une approbation avant toute exécution, prévenant les actions offensives non maîtrisées.

Conclusion

Le projet Pentest RAG est une solution d'ingénierie pragmatique qui comble le fossé entre la connaissance brute du pentest et l'exploitation actionnable. Ses forces tiennent à quatre partis pris de conception : la qualité des données (déduplication, nettoyage, OCR), la récupération orientée cible, la sortie structurée du LLM et l'intégration explicite avec les agents — au premier rang desquels Strix. En fournissant une source unique et fiable de guidage d'exploitation, livrée dans un format directement utilisable par une machine, il réduit significativement le temps de recherche, accroît la précision des tentatives d'exploitation et accélère l'ensemble de la phase offensive d'un test d'intrusion. Son déploiement conteneurisé le rend facile à exécuter, à partager et à intégrer dans les chaînes outillées de sécurité existantes, tandis que son moteur d'exploitation à trois niveaux, encadré par une validation humaine, illustre un souci constant de fiabilité et de maîtrise.

Références & Webographie

- [1] Documentation interne — « Pentest RAG Project Report » et « Fonctionnement du Pentest RAG ».
- [2] « Pentest RAG + Strix — Quick User Guide » (guide de démarrage).
- [3] Qdrant — base de données vectorielle. <https://qdrant.tech>
- [4] LangChain — framework d'orchestration LLM. <https://python.langchain.com>
- [5] Sentence-Transformers / Cross-Encoders. <https://www.sbert.net>
- [6] Nomic Embed — nomic-ai/nomic-embed-text-v1. <https://huggingface.co/nomic-ai>
- [7] FastAPI — framework d'API Python. <https://fastapi.tiangolo.com>
- [8] Ollama — exécution locale de LLM. <https://ollama.com>
- [9] OpenRouter — passerelle multi-LLM cloud. <https://openrouter.ai>
- [10] Metasploit Framework & pymetasploit3. <https://www.metasploit.com>
- [11] Exploit-DB / SearchSploit. <https://www.exploit-db.com>
- [12] Tesseract OCR. <https://github.com/tesseract-ocr/tesseract>
- [13] Projet Strix — agent de pentest autonome. <https://github.com/usestrix/strix>